Timer — Metod ve Mimari Dokümantasyonu

Bu doküman, Timer projesindeki tüm Python modüllerinin mimarisini, veri modellerini, sınıflarını ve fonksiyonlarını detaylandırmaktadır.

1. Mimari Genel Bakış

Timer, kullanıcıların birden fazla geri sayım zamanlayıcısını eşzamanlı olarak yönetebildiği, doğal dil ifadeleriyle akıllı alarmlar oluşturabildiği, periyodik/tekrarlı hatırlatıcılar kurabildiği, alarm seslerini özelleştirebildiği ve bekleme sürelerinde gömülü mini oyunlar (Sudoku, Tetris) oynayabildiği kapsamlı bir masaüstü zaman yönetim platformudur.

Uygulama, Olay Güdümlü (Event-Driven) Modüler Masaüstü Mimarisi ve MVC (Model-View-Controller) prensipleri doğrultusunda geliştirilmiştir:

- **Model Katmanı:** Hatırlatıcı veri yapıları (`Hatirlatici`, `HatirlaticiSistemi`), Tetris oyun mantığı ve parça matrisleri (`TetrisPiece`), Sudoku tahta ve bulmaca üretim algoritmaları (`SudokuUretici`), zamanlayıcı ayarları ve geçmiş kayıt modelleri.
- **Görünüm (View) Katmanı:** PyQt5 tabanlı ana pencere (`ZamanlayiciUygulamasi`), dinamik kart bileşenleri (`ZamanlayiciWidget`, `ClickableFrame`), yardımcı diyalog pencereleri (`YeniZamanlayiciDialog`, `AlarmDialog`, `YardimPenceresi`, `IlerlemeDialog`, `HatirlaticiYonetimPenceresi`, `LogViewerDialog`, `DosyaAnaliziPenceresi`, `SudokuOyunu`, `TetrisOyunu`).
- **Kontrolör / İş Mantığı (Controller):** Arka plan doğal dil iş parçacığı (`NaturalLanguageTimerThread`), hatırlatıcı tetikleyici ve koordinatör (`HatirlaticiManager`), ses ve medya denetleyici (`QMediaPlayer`), log yöneticisi (`TimerLogger`), genel yardımcılar (`TimerHelpers`).

Uygulamanın temel kütüphaneleri `PyQt5` (QtWidgets, QtCore, QtGui, QtMultimedia), `dateparser` (doğal dil zaman analizi), `datetime` & `calendar` (zaman hesaplamaları) ve `json` (kalıcı veri depolama) bileşenlerinden oluşur. Bileşenler arası veri akışı PyQt5'in Signal-Slot mimarisi ve QTimer tabanlı periyodik yoklama mekanizmaları ile asenkron şekilde koordine edilir.

2. Klavye Kısayolları ve Etkileşimler

Kısayol	İşlev
`F`	Favori zamanlayıcılar listesini açar veya kapatır
`G`	Geçmiş zamanlayıcılar listesini açar veya kapatır
`Escape`	Aktif pencereyi veya açık olan diyalog kutusunu kapatır
`Enter`	Doğal dil giriş kutusundaki ifadeyi analiz ederek zamanlayıcıyı başlatır
`Çift Tıklama`	Zamanlayıcı kartına (`ClickableFrame`) çift tıklandığında düzenleme diyaloğunu açar
`Sağ Tıklama`	Liste ve kart öğeleri üzerinde içerik menüsünü (Düzenle, Sil, Başlat) açar
`Ctrl + S`	Log görüntüleyici penceresinde kayıtları kaydeder / dışa aktarır
`Sol / Sağ Ok`	Tetris oyununda aktif parçayı sola / sağa hareket ettirir
`Yukarı Ok`	Tetris oyununda aktif parçayı saat yönünde döndürür
`Aşağı Ok`	Tetris oyununda hızlı düşürme (Soft Drop) yapar
`Boşluk (Space)`	Tetris oyununda anında düşürme (Hard Drop) yapar
`C` veya `Shift`	Tetris oyununda aktif parçayı bekletmeye (Hold) alır
`P`	Tetris veya Sudoku oyununu duraklatır / devam ettirir
`R`	Tetris oyununu yeniden başlatır
`1`, `2`, `3`	Tetris jokerlerini etkinleştirir (Satır silme, blok değiştirme, yavaşlatma)
`Yön Tuşları`	Sudoku tahtasında hücreler arasında gezinmeyi sağlar
`1 - 9 Rakamlar`	Sudoku tahtasında seçili hücreye sayı yerleştirir
`Backspace / Del`	Sudoku tahtasında seçili hücredeki sayıyı temizler
`H`	Sudoku oyununda ipucu (Hint) verir
`N`	Yeni Sudoku oyunu başlatır

3. Veritabanı ve Şema Yapısı

Uygulama, verilerini yerel dosya sistemindeki JSON yapılandırma dosyalarında ve metin tabanlı log dosyalarında saklar:

- `zamanlayici_ayarlar.json` **(Merkezi Ayarlar, Geçmiş ve Hatırlatıcılar):**
  - `son_sure`: Son ayarlanan varsayılan zamanlayıcı süresi (*integer*).
  - `gecmis`: Tamamlanan zamanlayıcıların geçmiş kayıtları (*array*). Her kayıt `tarih` (*string*), `sure` (*integer*), `aciklama` (*string*), `alarm` (*string*), `tekrar_toplam_sayi` (*integer*), `tekrar_mevcut_calisma` (*integer*), `tekrar_araligi_dakika` (*integer*), `ozel_saat_aktif_ilk_calisma` (*boolean*), `ozel_saat_str` (*string/null*) alanlarını içerir.
  - `favoriler`: Kullanıcı tarafından kaydedilen şablon zamanlayıcılar (*array* - `gecmis` ile aynı nesne yapısı).
  - `hatirlaticilar`: Hatırlatıcı nesneleri dizisi (*array*). Her kayıt `id` (*integer*), `tarih` (*string - ISO 8601*), `saat` (*string*), `not_metni` (*string*), `yapildi` (*boolean*), `olusturma_zamani` (*string*), `tekrarlama_tipi` (*string: "yok", "gun", "hafta", "ay"*), `tekrarlama_araligi` (*integer*), `son_tekrar_tarihi` (*string/null*), `hafta_gunu` (*integer/null*), `ertelendi` (*boolean*) alanlarını barındırır.
- `zamanlayici_log.txt` **ve** `log_favori_search.txt` **(Sistem Günlükleri):**
  - Mikro saniye hassasiyetli log formatı: `[YYYY-MM-DD HH:MM:SS.mmm] [SEVIYE] [MODUL:SATIR] Mesaj`.
- `tetris_scores.json` **(Tetris Skorları):**
  - Kayıt dizisi (*array*): `name` (*string*), `score` (*integer*), `level` (*integer*), `lines` (*integer*), `play_time` (*string*), `date` (*string*).
- `sudoku_puanlar.json` **(Sudoku Skorları):**
  - Zorluk seviyelerine göre gruplandırılmış nesne (`Kolay`, `Orta`, `Zor`). Dizi elemanları: `isim` (*string*), `puan` (*integer*), `sure` (*integer*), `ipucu` (*integer*), `silme` (*integer*), `kontrol` (*integer*), `hata` (*integer*), `oyun_sayisi` (*integer*), `tarih` (*string*).

MODÜL VE YÖNTEM DETAYLARI

__temp.py

Modül Tanımı: Hatırlatıcı yöneticisi için geçici ve yardımcı bileşenleri barındıran modül.

`HatirlaticiManager` (Sınıf)

Hatırlatıcı sisteminin UI ve yönetim işlevlerini sağlayan yardımcı sınıf.

- `__init__(self, parent_app)`
  - **İşlevi:** HatirlaticiManager sınıfını ve gerekli bileşenlerini ilklendirir.
  - **Parametreler:**
    - `parent_app`: Ana uygulama penceresi (`ZamanlayiciUygulamasi`) referansı.
- `yeni_hatirlatici_ekle(self)`
  - **İşlevi:** Yeni hatırlatıcı ekleme diyaloğunu aç.
- `hatirlatici_sag_tik_menu(self, position)`
  - **İşlevi:** Tüm hatırlatıcılar listesi için sağ tık menüsü.
  - **Parametreler:**
    - `position`: Fare tıklamasının gerçekleştiği koordinat noktası (QPoint).
- `get_hatirlatici_from_sorted_list(self, sorted_index)`
  - **İşlevi:** Sıralı listeden gerçek hatırlatıcı objesini döndür - ID tabanlı.
  - **Parametreler:**
    - `sorted_index`: Sıralı listedeki hedef öğe indeksi.
- `hatirlatici_duzenle_by_object(self, hatirlatici)`
  - **İşlevi:** Hatırlatıcı objesini kullanarak düzenleme.
  - **Parametreler:**
    - `hatirlatici`: Düzenlenecek Hatirlatici nesnesi referansı.
- `hatirlatici_kontrol(self)`
  - **İşlevi:** Zamanı gelen hatırlatıcıları kontrol et.
- `hatirlatici_bildirim_goster(self, hatirlatici)`
  - **İşlevi:** Hatırlatıcı bildirimini göster.
  - **Parametreler:**
    - `hatirlatici`: İşlem yapılacak Hatirlatici nesnesi referansı.
- `hatirlatici_ses_cal(self, ses_dosyasi)`
  - **İşlevi:** Hatırlatıcı için ses çal.
  - **Parametreler:**
    - `ses_dosyasi`: Çalınacak ses veya alarm dosyasının adı.

__test_export.py

Modül Tanımı: Dosya analiz penceresi dışa aktarma (export) işlevselliğini doğrulayan test betiği.

Bağımsız Fonksiyonlar

- `test_export_functionality()`
  - **İşlevi:** Test the export functionality by opening the file analyzer window.

---

dialog_classes.py

Modül Tanımı: Zamanlayıcı oluşturma/düzenleme, alarm bildirimi, yardım ve ilerleme diyalog pencerelerini içeren PyQt5 arayüz modülü.

`ClickableFrame` (Sınıf)

Çift tıklanabilir frame sınıfı.

- `__init__(self, timer_id_str, parent)`
  - **İşlevi:** ClickableFrame sınıfını ve gerekli bileşenlerini ilklendirir.
  - **Parametreler:**
    - `timer_id_str`: Zamanlayıcının metin formatındaki tekil kimlik değeri.
    - `parent`: Üst pencere veya taşıyıcı arayüz bileşeni referansı.
- `mouseDoubleClickEvent(self, event)`
  - **İşlevi:** Sol fare tuşu ile çift tıklama yapıldığında ilgili zamanlayıcı kimliğini (timer_id_str) taşıyan doubleClicked sinyalini yayar.
  - **Parametreler:**
    - `event`: Tkinter fare veya klavye olay nesnesi (Event).

`YeniZamanlayiciDialog` (Sınıf)

Yeni zamanlayıcı oluşturma ve düzenleme dialogu.

- `__init__(self, parent, alarm_dosyalari, varsayilan_sure, veri_klasoru, zamanlayici_to_edit, is_editing_favorite)`
  - **İşlevi:** YeniZamanlayiciDialog sınıfını ve gerekli bileşenlerini ilklendirir.
  - **Parametreler:**
    - `parent`: Üst pencere veya taşıyıcı arayüz bileşeni referansı.
    - `alarm_dosyalari`: Kullanılabilir alarm ses dosyaları listesi.
    - `varsayilan_sure`: Varsayılan zamanlayıcı süresi (dakika).
    - `veri_klasoru`: Uygulama veri dosyalarının saklandığı dizin.
    - `zamanlayici_to_edit`: Düzenlenmekte olan zamanlayıcı nesnesi referansı.
    - `is_editing_favorite`: Düzenlenen kaydın favori şablonu olup olmadığını belirten bayrak.

- `arayuz_olustur(self)`
  - **İşlevi:** Dialog arayüzünü oluştur.
- `favorilere_ekle(self)`
  - **İşlevi:** Mevcut ayarları favorilere ekle.
- `degerleri_doldur(self, varsayilan_sure)`
  - **İşlevi:** Form değerlerini doldur.
  - **Parametreler:**
    - `varsayilan_sure`: Varsayılan zamanlayıcı süresi (dakika).
- `tekrar_ayarlarini_guncelle(self, deger)`
  - **İşlevi:** Tekrar sayısı değiştiğinde tekrar aralığı spinner'ını etkinleştir/devre dışı bırak.
  - **Parametreler:**
    - `deger`: Hücreye yerleştirilen sayısal değer.
- `alarm_zamani_durumu_degisti(self, state)`
  - **İşlevi:** Alarm zamanı checkbox durumu değiştiğinde QTimeEdit'i etkinleştir/devre dışı bırak.
  - **Parametreler:**
    - `state`: Bileşenin veya onay kutusunun durum değeri (Qt.CheckState).
- `alarm_secimi_degisti(self)`
  - **İşlevi:** Alarm seçimi değiştiğinde çalmayı durdur.
- `alarm_cal_durdur(self)`
  - **İşlevi:** Seçili alarmı (veya çoklu seçimde rastgele birini) çal veya durdur
- `oynatici_durum_degisti(self, durum)`
  - **İşlevi:** Medya oynatıcı durumu değiştiğinde çağrılır.
  - **Parametreler:**
    - `durum`: İşlem veya bağlantı durumunu belirten metin ya da bayrak.
- `getValues(self)`
  - **İşlevi:** Form değerlerini döndür
- `closeEvent(self, event)`
  - **İşlevi:** Dialog kapatıldığında çalmayı durdur.
  - **Parametreler:**
    - `event`: Tkinter fare veya klavye olay nesnesi (Event).
- `favorileri_comboboxa_yukle(self)`

- **İşlevi:** Favori zamanlayıcıları combobox'a yükler.

- `sonra_baslat_durumu_degisti(self, state)`
  - **İşlevi:** Sonra Başlat seçeneği değiştiğinde alarm saati ve combobox durumunu ayarlar.
  - **Parametreler:**
    - `state`: Onay kutusunun (QCheckBox) yeni seçim durumu (Qt.Checked veya Qt.Unchecked).

`CheckableComboBox` (Sınıf)

Birden fazla öğenin checkbox ile işaretlenebildiği özel ComboBox sınıfı.

- `__init__(self, parent)`
  - **İşlevi:** Çoklu seçimli ComboBox bileşenini başlatır.
  - **Parametreler:**
    - `parent`: Üst pencere veya taşıyıcı arayüz bileşeni referansı.

- `eventFilter(self, obj, event)`
  - **İşlevi:** Fare tıklamalarını yakalayarak öğelerin işaretlenmesini ve menünün açık kalmasını sağlar.
  - **Parametreler:**
    - `obj`: Olayın gerçekleştiği hedef Qt nesnesi (lineEdit veya viewport).
    - `event`: Yakalanan Qt fare veya klavye olay nesnesi.

- `_veri_degisti(self, top_left, bottom_right, roles)`
  - **İşlevi:** Model verisi değiştiğinde görünen metni günceller.
  - **Parametreler:**
    - `top_left`: Değişen model aralığının sol üst indeks nesnesi.
    - `bottom_right`: Değişen model aralığının sağ alt indeks nesnesi.
    - `roles`: Güncellenen model veri rollerinin listesi.

- `addCheckableItem(self, text, data, checked)`
  - **İşlevi:** ComboBox içerisine işaretlenebilir yeni bir öğe ekler.
  - **Parametreler:**
    - `text`: Açılır listede görüntülenecek öğe metni.
    - `data`: Öğeyle ilişkilendirilen gerçek veri (ses dosyası adı).
    - `checked`: Öğenin başlangıçta işaretli olup olmayacağını belirten mantıksal değer.

- `checkedDataList(self)`
  - **İşlevi:** İşaretlenmiş olan tüm öğelerin veri (dosya adı) listesini döndürür.

- `setCheckedDataList(self, alarm_verisi)`

    - **İşlevi:** Verilen tekil alarm adını veya alarm listesini ComboBox içinde işaretler.
    - **Parametreler:**
        - `alarm_verisi`: İşaretlenecek tekil alarm dosya adı (`str`) veya dosya adları listesi (`list`).
- `metni_guncelle(self)`
    - **İşlevi:** İşaretli öğelere göre ComboBox üzerindeki özet metni ve ipucunu günceller.

---

ti_dialogs.py

Modül Tanımı: İlerleme göstergesi, yardım penceresi ve alarm bildirim penceresini içeren PyQt5 diyalog sınıfları modülü.

`IlerlemeDialog` (Sınıf)

İlerleme gösterge dialogu.

- `__init__(self, parent, baslik)`
    - **İşlevi:** IlerlemeDialog sınıfını ve gerekli bileşenlerini ilklendirir.
    - **Parametreler:**
        - `parent`: Üst pencere veya taşıyıcı arayüz bileşeni referansı.
        - `baslik`: Diyalog veya bildirim penceresinin başlık metni.
- `mesaj_guncelle(self, mesaj)`
    - **İşlevi:** İlerleme mesajını güncelle.
    - **Parametreler:**
        - `mesaj`: Kullanıcıya gösterilecek uyarı veya bilgi mesajı.
- `ilerleme_guncelle(self, yuzde, mesaj)`
    - **İşlevi:** İlerleme çubuğunu ve mesajı güncelle.
    - **Parametreler:**
        - `yuzde`: İlerleme çubuğunda gösterilecek tamamlanma yüzdesi (0-100).
        - `mesaj`: Kullanıcıya gösterilecek uyarı veya bilgi mesajı.
- `islem_sayaci_guncelle(self, mevcut, toplam)`
    - **İşlevi:** İşlem sayacını güncelle ve yüzde hesapla.
    - **Parametreler:**
        - `mevcut`: Tamamlanan işlem adımı sayısı.
        - `toplam`: Toplam işlem adımı sayısı.
- `iptal_et(self)`

- **İşlevi:** İptal işlemini başlat.

- `closeEvent(self, event)`
  - **İşlevi:** Dialog kapatıldığında iptal bayrağını ayarla.
  - **Parametreler:**
    - `event`: Tkinter fare veya klavye olay nesnesi (Event).

`YardimPenceresi` (Sınıf)

Yardım penceresi.

- `__init__(self, parent, baslik, icerik, html_dosya_yolu)`
  - **İşlevi:** YardimPenceresi sınıfını ve gerekli bileşenlerini ilklendirir.
  - **Parametreler:**
    - `parent`: Üst pencere veya taşıyıcı arayüz bileşeni referansı.
    - `baslik`: Diyalog veya bildirim penceresinin başlık metni.
    - `icerik`: Pencerede görüntülenecek metin veya HTML içeriği.
    - `html_dosya_yolu`: Yüklenecek yardım veya kılavuz HTML dosyasının yolu.
- `_fix_image_paths(self, html, html_path)`
  - **İşlevi:** HTML içindeki

    

    yollarını mutlak dosya yoluna çevirir.
  - **Parametreler:**
    - `html`: Görüntülenecek HTML biçimli metin içeriği.
    - `html_path`: Açılacak HTML dosyasının dosya yolu.

`AlarmDialog` (Sınıf)

Alarm dialogu - QDialog'dan türetilmiş özel dialog.

- `__init__(self, parent, title, message, ses_dosyasi)`
  - **İşlevi:** AlarmDialog sınıfını ve gerekli bileşenlerini ilklendirir.
  - **Parametreler:**
    - `parent`: Üst pencere veya taşıyıcı arayüz bileşeni referansı.
    - `title`: Pencere veya diyalog başlık metni.
    - `message`: Kullanıcıya gösterilecek durum veya bildirim iletisi.
    - `ses_dosyasi`: Çalınacak ses veya alarm dosyasının adı.
- `arayuz_olustur(self, message)`
  - **İşlevi:** Dialog arayüzünü oluştur.

    - **Parametreler:**
        - `message`: Kullanıcıya gösterilecek durum veya bildirim iletisi.

- `sure_guncelle(self)`
    - **İşlevi:** Geçen süreyi güncelle.
- `center_dialog(self)`
    - **İşlevi:** Dialog'u ekranın ortasına yerleştir.
- `force_update(self)`
    - **İşlevi:** Pencereyi zorla güncelle.
- `sesi_baslat(self)`
    - **İşlevi:** Alarm sesini çalmaya başlar.
- `medya_durumu_degisti(self, durum)`
    - **İşlevi:** Medya durumu değiştiğinde döngü için kontrol.
    - **Parametreler:**
        - `durum`: İşlem veya bağlantı durumunu belirten metin ya da bayrak.
- `sesi_durdur(self)`
    - **İşlevi:** Alarm sesini durdur.
- `keyPressEvent(self, event)`
    - **İşlevi:** Klavye olaylarını yakala.
    - **Parametreler:**
        - `event`: Tkinter fare veya klavye olay nesnesi (Event).
- `closeEvent(self, event)`
    - **İşlevi:** Dialog kapatılırken sesi durdur.
    - **Parametreler:**
        - `event`: Tkinter fare veya klavye olay nesnesi (Event).
- `accept(self)`
    - **İşlevi:** Tamam düğmesine basıldığında.
- `reject(self)`
    - **İşlevi:** İptal veya X düğmesine basıldığında.

`FavoriGeriYukleDialog` (Sınıf)

Yedekten geri dönülecek favorileri listeleme ve onaylama penceresi.

- `__init__(self, favoriler, dosya_adi, parent)`
  - **İşlevi:** Favori geri yükleme penceresini başlatır ve arayüzü hazırlar.
  - **Parametreler:**
    - `favoriler`: Yedekten yüklenen favori sözlükleri listesi.
    - `dosya_adi`: Seçilen yedek dosyasının adı.
    - `parent`: Üst pencere referansı.
- `init_ui(self)`
  - **İşlevi:** Diyalog penceresinin bileşenlerini ve düzenini oluşturur.

sudoku_game.py

Modül Tanımı: PyQt5 tabanlı, tahta oluşturma, bulmaca üretimi, çözüm denetleme ve puan tablosu içeren Sudoku oyunu.

`SudokuOyunu` (Sınıf)

SudokuOyunu sınıfı, ilgili işlevleri ve süreçleri yönetir.

- `__init__(self, parent)`
  - **İşlevi:** SudokuOyunu sınıfını ve gerekli bileşenlerini ilklendirir.
  - **Parametreler:**
    - `parent`: Üst pencere veya taşıyıcı arayüz bileşeni referansı.
- `menu_olustur(self)`
  - **İşlevi:** Sudoku ana penceresinin üst menü çubuğunu (Yeni Oyun, Zorluk Seçimi, Puan Tablosu, Yardım) oluşturur.
- `keyPressEvent(self, event)`
  - **İşlevi:** Klavye olaylarını yakalayarak ilgili yönlendirme veya kısayol işlemlerini yürütür.
  - **Parametreler:**
    - `event`: Klavye olay nesnesi (QKeyEvent).
- `yeni_oyun(self, puan_sifirla)`
  - **İşlevi:** Yeni oyun başlatır ve oyun tahtasını hazırlar.
  - **Parametreler:**
    - `puan_sifirla`: Puan sayacının sıfırlanıp sıfırlanmayacağını belirten bayrak.

- `sudoku_olustur(self, zorluk)`
  - **İşlevi:** Sudoku tahtası oluşturur ve çözümünü saklar.
  - **Parametreler:**
    - `zorluk`: Oyun zorluk seviyesi ("Kolay", "Orta", "Zor").
- `sudoku_coz(self, tahta)`
  - **İşlevi:** Sudoku çözme algoritmasını çalıştırır.
  - **Parametreler:**
    - `tahta`: Sudoku tahtası matris verisi (9x9 liste).
- `bos_hucre_bul(self, tahta)`
  - **İşlevi:** Tahta üzerindeki ilk boş hücreyi bulur.
  - **Parametreler:**
    - `tahta`: Sudoku tahtası matris verisi (9x9 liste).
- `sayi_uygun_mu(self, tahta, sayi, pos)`
  - **İşlevi:** Sayının belirtilen konuma uygun olup olmadığını denetler.
  - **Parametreler:**
    - `tahta`: Sudoku tahtası matris verisi (9x9 liste).
    - `sayi`: İşlem yapılacak sayısal değer (1-9 arası rakam veya miktar).
    - `pos`: Hücre konumu koordinatı (satir, sutun).
- `olasi_sayilari_bul(self, satir, sutun)`
  - **İşlevi:** Bir hücreye yerleştirilebilecek olası sayıları bulur.
  - **Parametreler:**
    - `satir`: Tahta veya ızgara üzerindeki satır indeksi.
    - `sutun`: Tahta veya ızgara üzerindeki sütun indeksi.
- `sure_guncelle(self)`
  - **İşlevi:** Sudoku oyununun geçen süresini her saniye günceller ve arayüzdeki süre gösterge etiketine yansıtır.
- `cozumu_kontrol_et(self)`
  - **İşlevi:** Tahtadaki mevcut sayıların doğruluğunu ve oyunun tamamlanma durumunu denetler.
- `skor_tablosuna_girebilir_mi_kontrol(self, yeni_puan)`
  - **İşlevi:** Yeni puanın skor tablosuna girip girmeyeceğini kontrol eder.
  - **Parametreler:**
    - `yeni_puan`: Skor tablosuna işlenecek yeni puan değeri.
- `oyunu_bitir(self)`

    - **İşlevi:** Aktif oyunu sonlandırır ve skor kontrolü yapar.

- `oyun_bitti(self, mesaj)`
    - **İşlevi:** Silme hakkı veya hata limiti bittiğinde oyunu bitirir ve puan tablosunu denetler.
    - **Parametreler:**
        - `mesaj`: Kullanıcıya gösterilecek uyarı veya bilgi mesajı.
- `puan_kaydet(self, isim, puan, sure)`
    - **İşlevi:** Kazanılan puanı ve istatistikleri skor tablosuna kaydeder.
    - **Parametreler:**
        - `isim`: Oyuncu kullanıcı adı.
        - `puan`: Oyuncunun kazandığı puan miktarı.
        - `sure`: Geçen süre değeri.
- `zorluk_degisti(self, yeni_zorluk)`
    - **İşlevi:** Zorluk seviyesi değiştiğinde yeni oyunu başlatır veya onay ister.
    - **Parametreler:**
        - `yeni_zorluk`: Seçilen yeni oyun zorluk derecesi.
- `puan_tablosunu_goster(self)`
    - **İşlevi:** Skor ve derece tablosunu gösterir.
- `puan_tablosu_sag_tus_menusu(self, pos, zorluk, tablo)`
    - **İşlevi:** Puan tablosu üzerinde sağ tıklama menüsünü açar.
    - **Parametreler:**
        - `pos`: Fare imlecinin tıklandığı yerel koordinat noktası.
        - `zorluk`: Oyun zorluk seviyesi ("Kolay", "Orta", "Zor").
        - `tablo`: Puanların listelendiği QTableWidget bileşeni.
- `puan_kaydi_sil(self, zorluk, satir_index)`
    - **İşlevi:** Puan tablosundan belirtilen kaydı siler.
    - **Parametreler:**
        - `zorluk`: Oyun zorluk seviyesi ("Kolay", "Orta", "Zor").
        - `satir_index`: Tablodaki hedef satır indeksi.
- `yardim_goster(self)`
    - **İşlevi:** Sudoku kurallarını ve yardım penceresini gösterir.

---

ti_sudoku_board.py

Modül Tanımı: Sudoku hücre stilleri, tahta görselleştirmesi ve hücre etkileşimlerini yöneten Mixin modülü.

`SudokuBoardMixin` (Sınıf)

Sudoku tahta görselleştirmesi, hücre stilleri ve kullanıcı etkileşim mixini.

- `normal_hucre_stili(self, satir, sutun)`
  - **İşlevi:** Hücrenin temel CSS stilini döndürür.
  - **Parametreler:**
    - `satir`: Tahta veya ızgara üzerindeki satır indeksi.
    - `sutun`: Tahta veya ızgara üzerindeki sütun indeksi.
- `hucre_vurgulu_mu(self, satir, sutun)`
  - **İşlevi:** Hücrenin vurgulanması gerekip gerekmediğini denetler.
  - **Parametreler:**
    - `satir`: Tahta veya ızgara üzerindeki satır indeksi.
    - `sutun`: Tahta veya ızgara üzerindeki sütun indeksi.
- `hucre_stilini_guncelle(self, satir, sutun, secili)`
  - **İşlevi:** Hücrenin stilini durumuna göre günceller.
  - **Parametreler:**
    - `satir`: Tahta veya ızgara üzerindeki satır indeksi.
    - `sutun`: Tahta veya ızgara üzerindeki sütun indeksi.
    - `secili`: Hücrenin seçili olma durumu bayrağı.
- `hucreyi_guncelle(self, satir, sutun)`
  - **İşlevi:** Hücrenin içeriğini (sayı veya notlar) görsel olarak günceller.
  - **Parametreler:**
    - `satir`: Tahta veya ızgara üzerindeki satır indeksi.
    - `sutun`: Tahta veya ızgara üzerindeki sütun indeksi.
- `tahtayi_guncelle(self)`
  - **İşlevi:** Tahtayı görsel olarak baştan sona günceller.
- `tum_hucreleri_guncelle(self)`
  - **İşlevi:** Tüm hücrelerin stillerini vurgulama durumlarına göre günceller.
- `hucre_secildi(self, satir, sutun)`
  - **İşlevi:** Bir hücre seçildiğinde seçimi günceller ve tahtayı vurgular.

    - **Parametreler:**
        - `satir`: Tahta veya ızgara üzerindeki satır indeksi.
        - `sutun`: Tahta veya ızgara üzerindeki sütun indeksi.

- `ok_tusu_hareketi(self, key)`
    - **İşlevi:** Ok tuşları ile tahta üzerindeki seçili hücreyi hareket ettirir.
    - **Parametreler:**
        - `key`: Basılan yön tuşu kodu (Qt.Key).

- `gecmise_ekle(self, satir, sutun, eski_deger, yeni_deger)`
    - **İşlevi:** Yapılan bir hamleyi geri alma geçmişine ekler.
    - **Parametreler:**
        - `satir`: Tahta veya ızgara üzerindeki satır indeksi.
        - `sutun`: Tahta veya ızgara üzerindeki sütun indeksi.
        - `eski_deger`: Değiştirilmeden önceki hücre değeri.
        - `yeni_deger`: Hücreye girilen yeni sayısal değer.

- `geri_al(self)`
    - **İşlevi:** Son hamleyi geri alır.

- `ileri_al(self)`
    - **İşlevi:** Geri alınan hamleyi tekrar uygular.

- `otomatik_kontrol(self)`
    - **İşlevi:** Tüm hücreler doldurulduğunda otomatik çözüm kontrolünü tetikler.

- `ipucu_goster(self)`
    - **İşlevi:** Rastgele bir boş hücreye doğru sayıyı yerleştirir ve puan düşürür.

- `sag_tus_menusu(self, satir, sutun)`
    - **İşlevi:** Hücreye sağ tıklandığında olası sayıları listeleyen menüyü açar.
    - **Parametreler:**
        - `satir`: Tahta veya ızgara üzerindeki satır indeksi.
        - `sutun`: Tahta veya ızgara üzerindeki sütun indeksi.

- `sayi_sec(self, satir, sutun, sayi)`
    - **İşlevi:** Sağ tık menüsünden seçilen sayıyı hücreye yerleştirir.
    - **Parametreler:**
        - `satir`: Tahta veya ızgara üzerindeki satır indeksi.
        - `sutun`: Tahta veya ızgara üzerindeki sütun indeksi.
        - `sayi`: Seçilen sayısal değer.

- `ilgili_notlari_temizle(self, satir, sutun, sayi)`
  - **İşlevi:** Sayı girildiğinde aynı satır, sütun ve 3x3 bölgedeki ilgili notları temizler.
  - **Parametreler:**
    - `satir`: Tahta veya ızgara üzerindeki satır indeksi.
    - `sutun`: Tahta veya ızgara üzerindeki sütun indeksi.
    - `sayi`: Girilen sayısal değer.
- `tum_notlari_temizle(self)`
  - **İşlevi:** Tahtadaki tüm notları topluca temizler.
- `not_modu_degistir(self)`
  - **İşlevi:** Not modunu açar veya kapatır.

---

ti_sudoku_dialogs.py

Modül Tanımı: Sudoku skor tablosu, skor kaydı ve yardım pencerelerini yöneten modül.

Bağımsız Fonksiyonlar

- `get_puan_dosyasi(dosya_yolu)`
  - **İşlevi:** Puan kayıtlarının tutulduğu JSON dosyasının mutlak yolunu döner.
  - **Parametreler:**
    - `dosya_yolu`: İsteğe bağlı özel dosya yolu.
- `skor_tablosuna_girebilir_mi_kontrol(zorluk, yeni_puan, dosya_yolu)`
  - **İşlevi:** Yeni puanın skor tablosuna girmeye hak kazanıp kazanmadığını denetler.
  - **Parametreler:**
    - `zorluk`: Oyun zorluk derecesi.
    - `yeni_puan`: Oyuncunun kazandığı puan.
    - `dosya_yolu`: İsteğe bağlı özel dosya yolu.
- `puan_kaydet(zorluk, isim, puan, sure, ipucu_sayisi, silme_sayisi, kontrol_sayisi, hata_sayisi, oynanan_oyun_sayisi, dosya_yolu)`
  - **İşlevi:** Oyuncu skorunu ve istatistiklerini JSON dosyasına kaydeder.
  - **Parametreler:**
    - `zorluk`: Oyun zorluk derecesi.
    - `isim`: Oyuncu kullanıcı adı.
    - `puan`: Kazanılan puan.
    - `sure`: Oyun süresi.

        - `ipucu_sayisi`: Kullanılan ipucu sayısı.
        - `silme_sayisi`: Kullanılan silme hakkı sayısı.
        - `kontrol_sayisi`: Yapılan kontrol sayısı.
        - `hata_sayisi`: Yapılan hata sayısı.
        - `oynanan_oyun_sayisi`: Oynanan oyun sayısı.
        - `dosya_yolu`: İsteğe bağlı özel dosya yolu.

- `puan_kaydi_sil(parent, zorluk, satir_index, dosya_yolu)`
    - **İşlevi:** Belirtilen zorluk seviyesindeki puan kaydını listeden siler.
    - **Parametreler:**
        - `parent`: Üst pencere bileşeni referansı.
        - `zorluk`: Oyun zorluk derecesi.
        - `satir_index`: Silinecek kaydın satır indeksi.
        - `dosya_yolu`: İsteğe bağlı özel dosya yolu.

- `puan_tablosu_sag_tus_menusu(parent, pos, zorluk, tablo, on_sil_callback)`
    - **İşlevi:** Puan tablosu sağ tık silme menüsünü açar.
    - **Parametreler:**
        - `parent`: Üst pencere bileşeni referansı.
        - `pos`: Fare imlecinin tıklandığı yerel koordinat noktası.
        - `zorluk`: Oyun zorluk derecesi.
        - `tablo`: Tablo bileşeni (QTableWidget).
        - `on_sil_callback`: Kayıt silindiğinde çağrılacak geri çağırma işlevi.

- `puan_tablosunu_goster(parent, on_sil_callback)`
    - **İşlevi:** Skor tablosu iletişim kutusunu oluşturup görüntüler.
    - **Parametreler:**
        - `parent`: Üst pencere bileşeni referansı.
        - `on_sil_callback`: Kayıt silindiğinde çağrılacak geri çağırma işlevi.

- `yardim_goster(parent)`
    - **İşlevi:** Nasıl oynanır yardım kılavuzunu görüntüler.
    - **Parametreler:**
        - `parent`: Üst pencere bileşeni referansı.

---

ti_sudoku_engine.py

Modül Tanımı: Sudoku temel hücre yapısı ve bulmaca oluşturma / çözme motoru.

`SudokuHucre` (Sınıf)

Sudoku hücresi veri yapısını, değerini ve notlarını tutar.

- `__init__(self)`
  - **İşlevi:** Sudoku hücresini varsayılan değerlerle ilklendirir.

Bağımsız Fonksiyonlar

- `bos_hucre_bul(tahta)`
  - **İşlevi:** Sudoku tahtasındaki ilk boş hücreyi bulur.
  - **Parametreler:**
    - `tahta`: Sudoku tahtası matris verisi (9x9 liste).
- `sayi_uygun_mu(tahta, sayi, pos)`
  - **İşlevi:** Bir sayının satır, sütun ve 3x3 bölge kurallarına göre uygunluğunu denetler.
  - **Parametreler:**
    - `tahta`: Sudoku tahtası matris verisi (9x9 liste).
    - `sayi`: Denetlenecek rakam (1-9).
    - `pos`: Hücre koordinatı (satir, sutun).
- `sudoku_coz(tahta)`
  - **İşlevi:** Backtracking algoritması ile Sudoku bulmacasını çözer.
  - **Parametreler:**
    - `tahta`: Sudoku tahtası matris verisi (9x9 liste).
- `sudoku_olustur(zorluk)`
  - **İşlevi:** Belirtilen zorluk derecesine göre yeni bir Sudoku tahtası ve çözümü üretir.
  - **Parametreler:**
    - `zorluk`: Oyun zorluk derecesi ("Kolay", "Orta", "Zor").
- `olasi_sayilari_bul(tahta, satir, sutun)`
  - **İşlevi:** Belirtilen hücreye gelebilecek uygun rakamların kümesini bulur.
  - **Parametreler:**
    - `tahta`: Sudoku tahtası matris verisi (9x9 liste).
    - `satir`: Hücre satır indeksi.
    - `sutun`: Hücre sütun indeksi.

---

sudoku_test.py

Modül Tanımı: Sudoku oyunu penceresini ve bileşenlerini başlatan test betiği.

---

ti_tetris_dialogs.py

Modül Tanımı: Tetris yüksek skor yöneticisi, skor tablosu ve yardım pencerelerini içeren modül.

`HighScoreManager` (Sınıf)

HighScoreManager sınıfı, ilgili işlevleri ve süreçleri yönetir.

- `__init__(self, filename)`
  - **İşlevi:** HighScoreManager sınıfını ve gerekli bileşenlerini ilklendirir.
  - **Parametreler:**
    - `filename`: Dosya adı veya hedef kayıt dosyası.
- `load_scores(self)`
  - **İşlevi:** tetris_scores.json dosyasından kaydedilmiş yüksek skor verilerini okur ve listeye yükler.
- `save_scores(self)`
  - **İşlevi:** Güncel yüksek skor listesini tetris_scores.json dosyasına kalıcı olarak kaydeder.
- `add_score(self, name, score, level, lines, play_time, date_time)`
  - **İşlevi:** Yeni oyun skorunu listeye ekler, puana göre sıralar ve en iyi 10 skoru muhafaza ederek kaydeder.
  - **Parametreler:**
    - `name`: Oyuncunun girdiği isim veya rumuz.
    - `score`: Oyunda elde edilen toplam puan.
    - `level`: Oyunun tamamlandığı veya ulaşılan seviye numarası.
    - `lines`: Temizlenen toplam satır adedi.
    - `play_time`: Toplam oynama süresi (DD:SS).
    - `date_time`: Skorun elde edildiği tarih ve zaman damgası.
- `is_high_score(self, score)`
  - **İşlevi:** Elde edilen skorun en yüksek 10 skor arasına girmeye hak kazanıp kazanmadığını denetler.
  - **Parametreler:**
    - `score`: Kontrol edilecek puan değeri.

`HighScoreDialog` (Sınıf)

HighScoreDialog sınıfı, ilgili işlevleri ve süreçleri yönetir.

- `__init__(self, scores, parent)`
  - **İşlevi:** HighScoreDialog sınıfını ve gerekli bileşenlerini ilklendirir.
  - **Parametreler:**
    - `scores`: Skor kayıtlarını içeren sözlük veya liste.
    - `parent`: Üst pencere veya taşıyıcı arayüz bileşeni referansı.
- `init_ui(self, scores)`
  - **İşlevi:** Tetris ana penceresinin arayüz bileşenlerini, gösterge panellerini, joker butonlarını ve tuş dinleyicilerini ilklendirir.
  - **Parametreler:**
    - `scores`: Skor kayıtlarını içeren sözlük veya liste.

`HelpDialog` (Sınıf)

HelpDialog sınıfı, ilgili işlevleri ve süreçleri yönetir.

- `__init__(self, parent)`
  - **İşlevi:** HelpDialog sınıfını ve gerekli bileşenlerini ilklendirir.
  - **Parametreler:**
    - `parent`: Üst pencere veya taşıyıcı arayüz bileşeni referansı.
- `init_ui(self)`
  - **İşlevi:** Tetris ana penceresinin arayüz bileşenlerini, gösterge panellerini, joker butonlarını ve tuş dinleyicilerini ilklendirir.

---

ti_tetris_board.py

Modül Tanımı: Tetris oyun tahtası mekaniklerini, parça rotasyonlarını ve önizleme bileşenini içeren modül.

`TetrisBoard` (Sınıf)

TetrisBoard sınıfı, ilgili işlevleri ve süreçleri yönetir.

- `__init__(self, parent)`
  - **İşlevi:** TetrisBoard sınıfını ve gerekli bileşenlerini ilklendirir.
  - **Parametreler:**

        - `parent`: Üst pencere veya taşıyıcı arayüz bileşeni referansı.
- `init_board(self)`
    - **İşlevi:** Tetris oyun tahtasını satır ve sütun boyutlarına göre boş hücrelerle ilklendirir.
- `start(self)`
    - **İşlevi:** Oyun tahtasını sıfırlar, ilk parçaları üretir ve oyun döngüsü zamanlayıcısını başlatır.
- `pause(self)`
    - **İşlevi:** Oyun zamanlayıcısını duraklatır veya duraklatılmış durumdaysa devam ettirir.
- `spawn_next_piece(self)`
    - **İşlevi:** Sıradaki parçayı tahtaya aktif parça olarak alır ve sonraki parçayı rastgele üretir.
- `spawn_new_piece(self)`
    - **İşlevi:** Yeni bir rastgele Tetris parçası oluşturur ve tahtanın tepe noktasına konumlandırır.
- `change_current_piece(self)`
    - **İşlevi:** F2 ile mevcut parçayı değiştir.
- `use_joker(self)`
    - **İşlevi:** F3 ile joker kullan - en alt satırı temizle.
- `check_joker_earn(self)`
    - **İşlevi:** Her 300 puanda joker kazan.
- `is_valid_position(self, piece, x, y)`
    - **İşlevi:** Verilen parçanın belirtilen (x, y) koordinatlarında tahta sınırları ve sabit bloklarla çakışıp çakışmadığını denetler.
    - **Parametreler:**
        - `piece`: Konumu denetlenecek TetrisPiece nesnesi.
        - `x`: Yatay eksendeki piksel veya nokta koordinatı.
        - `y`: Düşey eksendeki piksel veya nokta koordinatı.
- `rotate_piece(self)`
    - **İşlevi:** Aktif parçayı saat yönünde döndürür; sınır aşımı durumunda geçerli bir duvardan sektirme pozisyonu arar.
- `move_piece(self, dx)`
    - **İşlevi:** Aktif Tetris parçasını yatay eksende belirtilen miktar kadar kaydırır ve tahta sınırlarını denetler.
    - **Parametreler:**

    - `dx`: Parçanın yatay eksendeki hareket yönü ve miktarı (-1: Sol, 1: Sağ).
- `soft_drop(self)`
    - **İşlevi:** Aktif parçayı dikey olarak bir satır aşağı indirir; aşağı inemiyorsa parçayı tahtaya kilitler.
- `hard_drop(self)`
    - **İşlevi:** Aktif parçayı iniş izdüşümünün en alt noktasına anında düşürür ve kilitler.
- `hold_piece(self)`
    - **İşlevi:** Aktif parçayı bekletme (hold) haznesine alır veya daha önce bekletilen parça varsa onu tahtaya sürer.
- `get_ghost_position(self)`
    - **İşlevi:** Aktif parçanın mevcut konumundaki nihai iniş tahta koordinatlarını (hayalet blok izdüşümü) hesaplar.
- `lock_piece(self)`
    - **İşlevi:** İnişi tamamlanan parçanın bloklarını tahta ızgarasına kalıcı olarak yerleştirir ve satır temizleme kontrolünü başlatır.
- `clear_lines(self)`
    - **İşlevi:** Tamamen dolan yatay satırları tespit ederek siler, üst blokları aşağı indirir ve skora puan ekler.
- `get_play_time(self)`
    - **İşlevi:** Oyunun başlangıcından itibaren geçen toplam oynama süresini biçimlendirilmiş metin (DD:SS) olarak döndürür.
- `game_over_action(self)`
    - **İşlevi:** Oyun bittiğinde oyun döngüsü zamanlayıcısını durdurur, oyun sonu durumunu işaretler ve ana pencereye iletir.
- `timerEvent(self, event)`
    - **İşlevi:** QBasicTimer periyodik olayını yakalayarak aktif parçayı oyun seviyesine bağlı hızda otomatik aşağı kaydırır.
    - **Parametreler:**
        - `event`: Tkinter fare veya klavye olay nesnesi (Event).
- `paintEvent(self, event)`
    - **İşlevi:** Bileşenin grafiksel arayüzünü ve görsel öğelerini ekranda çizer.
    - **Parametreler:**
        - `event`: Tkinter fare veya klavye olay nesnesi (Event).

- `draw_square_v2(self, painter, x, y, size, color, is_falling)`
  - **İşlevi:** Kare çiz - düşen parça için neon glow efekt, yerleşikler için düz renk.
  - **Parametreler:**
    - `painter`: Çizim işlemlerini yürüten QPainter nesnesi.
    - `x`: Yatay eksendeki piksel veya nokta koordinatı.
    - `y`: Düşey eksendeki piksel veya nokta koordinatı.
    - `size`: Boyut, punto veya piksel büyüklüğü değeri.
    - `color`: Hex veya RGB formatında renk kodu (örn: "#FFCC00").
    - `is_falling`: Bloğun serbest düşüşte olup olmadığını belirten bayrak.
- `draw_square(self, painter, x, y, size, color, ghost)`
  - **İşlevi:** Belirtilen koordinata verilen renk ve boyut parametreleriyle 3D gölgeli veya hayalet blok karesi çizer.
  - **Parametreler:**
    - `painter`: Çizim işlemlerini gerçekleştiren QPainter nesnesi.
    - `x`: Yatay eksendeki piksel veya nokta koordinatı.
    - `y`: Düşey eksendeki piksel veya nokta koordinatı.
    - `size`: Boyut, punto veya piksel büyüklüğü değeri.
    - `color`: Hex veya RGB formatında renk kodu (örn: "#FFCC00").
    - `ghost`: Bloğun hayalet (yarı saydam) modda çizilip çizilmeyeceğini belirten bayrak.
- `square_width(self)`
  - **İşlevi:** Tahta genişliğine ve sütun sayısına göre tek bir hücrenin piksel genişliğini hesaplar.
- `square_height(self)`
  - **İşlevi:** Tahta yüksekliğine ve satır sayısına göre tek bir hücrenin piksel yüksekliğini hesaplar.
- `keyPressEvent(self, event)`
  - **İşlevi:** TetrisBoard için klavye olayları - BU METOD TetrisBoard SINIFINDA.
  - **Parametreler:**
    - `event`: Tkinter fare veya klavye olay nesnesi (Event).

`PreviewWidget` (Sınıf)

PreviewWidget sınıfı, ilgili işlevleri ve süreçleri yönetir.

- `__init__(self, parent)`
  - **İşlevi:** PreviewWidget sınıfını ve gerekli bileşenlerini ilklendirir.
  - **Parametreler:**
    - `parent`: Üst pencere veya taşıyıcı arayüz bileşeni referansı.
- `set_piece(self, piece_type)`

    - **İşlevi:** Sıradaki parça önizleme bileşenine görüntülenecek yeni Tetris parçasını atar ve yeniden çizdirir.
    - **Parametreler:**
        - `piece_type`: Görüntülenecek parçanın tipi veya şekil indeksi.

- `paintEvent(self, event)`
    - **İşlevi:** Bileşenin grafiksel arayüzünü ve görsel öğelerini ekranda çizer.
    - **Parametreler:**
        - `event`: Tkinter fare veya klavye olay nesnesi (Event).

---

tetris_game.py

Modül Tanımı: PyQt5 tabanlı, joker mekaniği, seviye sistemi ve yüksek skor kaydı içeren gelişmiş Tetris ana oyunu.

`TetrisOyunu` (Sınıf)

TetrisOyunu sınıfı, ilgili işlevleri ve süreçleri yönetir.

- `__init__(self)`
    - **İşlevi:** TetrisOyunu sınıfını ve gerekli bileşenlerini ilklendirir.

- `init_ui(self)`
    - **İşlevi:** Tetris ana penceresinin arayüz bileşenlerini, gösterge panellerini, joker butonlarını ve tuş dinleyicilerini ilklendirir.

- `showEvent(self, event)`
    - **İşlevi:** Pencere gösterildiğinde merkeze al.
    - **Parametreler:**
        - `event`: Tkinter fare veya klavye olay nesnesi (Event).

- `center_window(self)`
    - **İşlevi:** Pencereyi masaüstü ekranının merkez koordinatına yerleştirir.

- `update_score(self, score)`
    - **İşlevi:** Arayüzdeki güncel skor etiketini verilen yeni skor değeriyle günceller.
    - **Parametreler:**
        - `score`: Oyuncunun ulaştığı güncel puan değeri.

- `update_level(self, level)`

    - **İşlevi:** Arayüzdeki oyun seviyesi göstergesini günceller.
    - **Parametreler:**
        - `level`: Mevcut oyun zorluk seviyesi.

- `update_lines(self, lines)`
    - **İşlevi:** Arayüzdeki temizlenen toplam satır sayısı etiketini günceller.
    - **Parametreler:**
        - `lines`: Temizlenen toplam satır adedi.

- `update_jokers(self, jokers)`
    - **İşlevi:** Kalan joker haklarını (satır silme, blok değiştirme, yavaşlatma) göstergelerde günceller.
    - **Parametreler:**
        - `jokers`: Kalan joker miktarlarını içeren veri yapısı.

- `update_next_piece(self, piece_type)`
    - **İşlevi:** Sıradaki parça önizleme bileşenine yeni gelen parçayı aktarır.
    - **Parametreler:**
        - `piece_type`: Yeni gelen parçanın tipi veya şekli.

- `update_hold_piece(self, piece_type)`
    - **İşlevi:** Bekletilen (hold) parça kutusunu güncel parça ile yeniler.
    - **Parametreler:**
        - `piece_type`: Bekletmeye alınan parçanın tipi.

- `start_game(self)`
    - **İşlevi:** Yeni bir oyun oturumu başlatır, tahtayı ve sayaçları sıfırlar.

- `pause_game(self)`
    - **İşlevi:** Devam eden oyunu duraklatır veya duraklatılmış durumdaysa sürdürür.

- `restart_game(self)`
    - **İşlevi:** Oyunu başlangıç durumuna sıfırlayarak baştan başlatır.

- `handle_game_over(self)`
    - **İşlevi:** Oyun sona erdiğinde skorun yüksek skorlar listesine girip girmediğini kontrol eder ve kullanıcıya bildirir.

- `show_high_scores(self)`
    - **İşlevi:** Kaydedilmiş en yüksek skorları listeleyen iletişim kutusunu görüntüler.

- `show_help(self)`
    - **İşlevi:** Oyun kontrolleri ve kurallarını içeren yardım penceresini görüntüler.

- `keyPressEvent(self, event)`
  - **İşlevi:** Klavye olaylarını yakalayarak ilgili yönlendirme veya kısayol işlemlerini yürütür.
  - **Parametreler:**
    - `event`: Tkinter fare veya klavye olay nesnesi (Event).

---

tetris_test.py

Modül Tanımı: Tetris oyunu penceresini ve bileşenlerini başlatan test betiği.

---

ti_main.py

Modül Tanımı: Uygulamanın ana yürütülebilir modülü; çoklu zamanlayıcı arayüzü ve ana zamanlayıcı döngülerini koordine eder.

`ZamanlayiciUygulamasi` (Sınıf)

ZamanlayiciUygulamasi sınıfı, ana zamanlayıcı işlevlerini ve süreçleri yönetir.

- `__init__(self)`
  - **İşlevi:** ZamanlayiciUygulamasi sınıfını ve gerekli bileşenlerini ilklendirir.
- `natural_language_timer(self)`
  - **İşlevi:** Doğal dil girdi kutusuna yazılan metni arka plan iş parçacığıyla analiz ettirir ve zamanlayıcıyı başlatır.
- `hatirlatici_duzenle(self, index)`
  - **İşlevi:** Eski metod - artık kullanılmıyor, uyumluluk için bırakıldı.
  - **Parametreler:**
    - `index`: İşlem yapılacak öğenin dizin indeksi (int).
- `hatirlatici_sil(self, index)`
  - **İşlevi:** Eski metod - artık kullanılmıyor, uyumluluk için bırakıldı.
  - **Parametreler:**
    - `index`: İşlem yapılacak öğenin dizin indeksi (int).
- `gecmis_sag_tik_menu(self, position)`
  - **İşlevi:** Geçmiş listesi için sağ tık menüsü.

    - **Parametreler:**
        - `position`: Fare tıklamasının gerçekleştiği koordinat noktası (QPoint).

- `favori_sag_tik_menu(self, position)`
    - **İşlevi:** Favori listesi için sağ tık menüsü.
    - **Parametreler:**
        - `position`: Fare tıklamasının gerçekleştiği koordinat noktası (QPoint).
- `favori_duzenle(self, favori_indeks)`
    - **İşlevi:** Seçili favori zamanlayıcı kaydının adını, süresini veya alarm sesini düzenleme penceresini açar.
    - **Parametreler:**
        - `favori_indeks`: Düzenlenecek favori kaydının liste indeksi.
- `favoriye_ekle(self, gecmis_indeks)`
    - **İşlevi:** Geçmiş listesindeki veya formdaki bir zamanlayıcı ayarını kalıcı favoriler listesine ekler.
    - **Parametreler:**
        - `gecmis_indeks`: Favoriye dönüştürülecek geçmiş kaydının liste indeksi.
- `favoriden_sil(self, favori_indeks)`
    - **İşlevi:** Kullanıcının seçtiği favori zamanlayıcı şablonunu listeden ve kayıt dosyasından siler.
    - **Parametreler:**
        - `favori_indeks`: Silinecek favori kaydının liste indeksi.
- `favorileri_goster(self)`
    - **İşlevi:** Favori zamanlayıcılar kenar panelini görünür kılar veya gizler.
- `favori_listesini_guncelle(self)`
    - **İşlevi:** Bellekteki favori listesini arayüzdeki QListWidget bileşenine yeniden yükler.
- `favori_secimi_degisti(self)`
    - **İşlevi:** Favoriler listesinde yapılan seçim değişikliğine göre düzenleme ve silme butonlarının durumunu ayarlar.
- `secilen_favorileri_sil(self)`
    - **İşlevi:** Kullanıcı tarafından seçilmiş tüm favori kayıtlarını kullanıcıdan onay alarak topluca siler.
- `favori_zamanlayici_baslat(self, item)`
    - **İşlevi:** Seçili favori şablonunun süresini ve sesini kullanarak hemen yeni bir zamanlayıcı başlatır.
    - **Parametreler:**
        - `item`: Seçilen favori liste öğesi referansı (QListWidgetItem).

- `gecmis_secimi_degisti(self)`
  - **İşlevi:** Geçmiş listesinde bir kayıt seçildiğinde ilgili butonların aktiflik durumunu günceller.
- `secilen_gecmisi_sil(self)`
  - **İşlevi:** Geçmiş listesinden seçilen kayıtları temizler.
- `gecmisi_goster(self)`
  - **İşlevi:** Geçmiş zamanlayıcılar panelinin görünürlüğünü açıp kapatır.
- `gecmis_zamanlayici_baslat(self, item)`
  - **İşlevi:** Geçmiş listesindeki seçili kaydın parametreleriyle yeni bir zamanlayıcı başlatır.
  - **Parametreler:**
    - `item`: Seçilen geçmiş liste öğesi referansı (QListWidgetItem).
- `yeni_zamanlayici_baslat(self)`
  - **İşlevi:** Formdaki süre, açıklama, alarm sesi ve tekrar değerleriyle yeni bir zamanlayıcı oluşturup listeye ekler.
- `zamanlayici_beklet_devam(self, zamanlayici_id)`
  - **İşlevi:** Zamanlayıcıyı bekletir veya devam ettirir.
  - **Parametreler:**
    - `zamanlayici_id`: İşlem yapılacak zamanlayıcının tekil kimlik numarası.
- `zamanlayici_durdur(self, zamanlayici_id)`
  - **İşlevi:** İlgili kimliğe sahip zamanlayıcının geri sayımını durdurur ve listeden kaldırır.
  - **Parametreler:**
    - `zamanlayici_id`: Durdurulacak zamanlayıcının tekil kimlik değeri.
- `zamanlayici_tamamlandi(self, zamanlayici_id)`
  - **İşlevi:** Süresi dolan zamanlayıcı için alarm sesi çalar, görsel uyarı penceresi gösterir ve geçmişe kaydeder.
  - **Parametreler:**
    - `zamanlayici_id`: Süresi tamamlanan zamanlayıcının tekil kimlik değeri.
- `zamanlayici_guncelle(self)`
  - **İşlevi:** QTimer tarafından saniyede bir çağrılır; aktif zamanlayıcıların kalan sürelerini azaltır ve göstergeleri günceller.
- `sonra_baslat_tetikle(self, tamamlanan_zamanlayici)`
  - **İşlevi:** Bir zamanlayıcının tüm tekrarları bittiğinde onu bekleyen zamanlayıcıları başlatır.
  - **Parametreler:**

        - `tamamlanan_zamanlayici`: Tüm döngü ve alarmlarını başarıyla tamamlayan zamanlayıcı nesnesi.

- **`bekleme_zamanlayicisi_olustur(self, onceki_zamanlayici)`**
    - **İşlevi:** Tekrar aralığı için bekleme zamanlayıcısı oluşturur.
    - **Parametreler:**
        - `onceki_zamanlayici`: Düzenleme öncesindeki önceki zamanlayıcı verisi.
- **`sonraki_tekrari_baslat(self, onceki_zamanlayici_bilgileri)`**
    - **İşlevi:** Bekleme süresi dolduktan sonra asıl zamanlayıcıyı başlatır.
    - **Parametreler:**
        - `onceki_zamanlayici_bilgileri`: Düzenlenen zamanlayıcının önceki parametre sözlüğü.
- **`alarm_cal(self, aciklama, alarm_dosyasi)`**
    - **İşlevi:** Alarmı çal ve bildirim göster (çoklu ses seçilmişse rastgele birini çalar).
    - **Parametreler:**
        - `aciklama`: Zamanlayıcı veya görev açıklama metni.
        - `alarm_dosyasi`: Çalınacak tekil alarm ses dosyasının adı (`str`) veya arasından rastgele seçilecek ses dosyalarının listesi (`list`).
- **`bagli_favorileri_kontrol_et_ve_sor(self, baslatilan_favori, ziyaret_edilenler)`**
    - **İşlevi:** Başlatılan bir favori zamanlayıcıya 'Sonra Başlat' ile bağlı olan diğer favorileri kontrol eder ve kullanıcıya sorar.
    - **Parametreler:**
        - `baslatilan_favori`: Başlatılan favori verisi sözlüğü.
        - `ziyaret_edilenler`: Döngüsel bağlantıları önlemek için ziyaret edilen favori isimleri kümesi.

---

ti_app_window.py

Modül Tanımı: Ana uygulama penceresi çerçevesini, menü çubuğunu ve üst düzey pencere olaylarını yönetir.

`AnaUygulamaPenceresi` (Sınıf)

Ana uygulama ana pencere kapsayıcısı ve menü yönetimi sınıfı.

- **`__init__(self, zamanlayici_widget)`**
    - **İşlevi:** AnaUygulamaPenceresi sınıfını ve gerekli bileşenlerini ilklendirir.
    - **Parametreler:**

        - `zamanlayici_widget`: Ana pencere içerisine yerleştirilecek merkezi zamanlayıcı bileşeni.

- `menu_olustur(self)`
    - **İşlevi:** Menü çubuğunu oluştur.
- `yardim_goster(self)`
    - **İşlevi:** Yardım içeriğini timer-help.html dosyasından okuyup yeni bir pencerede gösterir.
- `program_hakkinda_goster(self)`
    - **İşlevi:** Uygulamanın sürüm, geliştirici ve genel amaç bilgilerini içeren hakkında iletişim penceresini görüntüler.
- `dosya_analizi_ac(self)`
    - **İşlevi:** Dosya analizi penceresini aç.
- `sudoku_ac(self)`
    - **İşlevi:** Sudoku penceresini aç.
- `tetris_ac(self)`
    - **İşlevi:** Tetris penceresini aç.
- `closeEvent(self, event)`
    - **İşlevi:** Pencere kapatılırken çalışan zamanlayıcıları ve kullanıcı tercihlerini ayar dosyasına kaydeder.
    - **Parametreler:**
        - `event`: Tkinter fare veya klavye olay nesnesi (Event).

---

ti_models.py

Modül Tanımı: Uygulamadaki zamanlayıcı ve kronometre veri modellerini barındırır.

`Kronometre` (Sınıf)

Kronometre sınıfı - yukarı doğru sayan zamanlayıcı.

- `__init__(self, id, isim, baslama_zamani)`
    - **İşlevi:** Kronometre sınıfını ve gerekli bileşenlerini ilklendirir.
    - **Parametreler:**
        - `id`: Kaydın tekil kimlik numarası.
        - `isim`: Kayıt, profil veya dosya adı.

        - `baslama_zamani`: Zamanlayıcının çalışmaya başladığı anlık zaman damgası.

- `get_formatted_time(self)`
    - **İşlevi:** Geçen süreyi saat:dakika:saniye formatında döndürür.
- `tur_ekle(self)`
    - **İşlevi:** Yeni tur ekler.
- `sifirla(self)`
    - **İşlevi:** Kronometreyi sıfırlar.
- `to_dict(self)`
    - **İşlevi:** Kronometreyi sözlük olarak kaydetmek için.
- `from_dict(cls, data)`
    - **İşlevi:** Sözlükten kronometre oluşturmak için.
    - **Parametreler:**
        - `data`: İşlenecek veya analiz edilecek veri kümesi veya nesnesi.

`Zamanlayici` (Sınıf)

Zamanlayici sınıfı, ilgili işlevleri ve süreçleri yönetir.

- `__init__(self, id, dakika_ayari, temel_aciklama, alarm, baslama_zamani_ilk_kurulum, tekrar_toplam_sayi, tekrar_mevcut_calisma, tekrar_araligi_dakika, ozel_saat_aktif_ilk_calisma, ozel_saat_str, sonra_baslat_aktif, beklenen_favori_aciklama)`
    - **İşlevi:** Zamanlayici sınıfını ve gerekli bileşenlerini ilklendirir.
    - **Parametreler:**
        - `id`: Kaydın tekil kimlik numarası.
        - `dakika_ayari`: Zamanlayıcı için belirlenen dakika süresi.
        - `temel_aciklama`: Zamanlayıcının kullanıcı tarafından girilen ana açıklama metni.
        - `alarm`: Alarm çaldığında yürütülecek ses dosyasının adı.
        - `baslama_zamani_ilk_kurulum`: Zamanlayıcının ilk kurulduğu andaki zaman damgası.
        - `tekrar_toplam_sayi`: Toplam tekrarlanma adedi.
        - `tekrar_mevcut_calisma`: Mevcut tekrar döngüsü numarası.
        - `tekrar_araligi_dakika`: Tekrarlar arasındaki bekleme aralığı (dakika).
        - `ozel_saat_aktif_ilk_calisma`: Belirli bir saatte çalışma özelliğinin aktifliği.
        - `ozel_saat_str`: Planlanan başlangıç saati dizgesi ("SS:DD").
        - `sonra_baslat_aktif`: Başka bir favori zamanlayıcının tamamlanmasının beklenip beklenmediği.

        - `beklenen_favori_aciklama`: Başlaması için tamamlanması beklenen favori zamanlayıcının açıklaması.

- `get_gorunen_aciklama(self)`
    - **İşlevi:** Görüntülenecek açıklamayı döndürür, tekrar bilgisini içerir.
- `to_dict(self)`
    - **İşlevi:** Zamanlayıcıyı sözlük olarak kaydetmek için.
- `from_dict(cls, data)`
    - **İşlevi:** Sözlükten zamanlayıcı oluşturmak için.
    - **Parametreler:**
        - `data`: İşlenecek veya analiz edilecek veri kümesi veya nesnesi.
- `secili_alarmlari_al(self)`
    - **İşlevi:** Zamanlayıcı için seçilmiş alarm dosyalarını liste olarak döndürür.
- `rastgele_alarm_sec(self)`
    - **İşlevi:** İşaretlenen alarm sesleri arasından rastgele bir tanesini seçip döndürür.
- `varsayilan_disi_alarm_var_mi(self)`
    - **İşlevi:** Varsayılan tekil alarm sesi dışında bir alarm veya çoklu seçim olup olmadığını döndürür.
- `get_alarm_kisa_etiket(self)`
    - **İşlevi:** Aktif zamanlayıcı kartında gösterilecek kısa alarm etiketini döndürür.
- `get_alarm_ipucu(self)`
    - **İşlevi:** Aktif zamanlayıcı kartındaki alarm etiketinin araç ipucunu (tooltip) döndürür.

Bağımsız Fonksiyonlar

- `format_alarm_bilgisi(alarm_degeri)`
    - **İşlevi:** Favori ve geçmiş listeleri için alarm bilgisini (.mp3 uzantısı olmadan) her zaman metin olarak biçimlendirir.
    - **Parametreler:**
        - `alarm_degeri`: Biçimlendirilecek tekil alarm dosya adı (`str`) veya çoklu alarm listesi (`list`).

---

ti_natural_parser.py

Modül Tanımı: Doğal dilde yazılan zaman/alarm ifadelerini ayrıştıran modül.

Bağımsız Fonksiyonlar

- `parse_natural_timer(text)`
  - **İşlevi:** Doğal dilde yazılan zaman/alarm ifadesini analiz ederek süre, hedef saat ve not sözlüğü döndürür.
  - **Parametreler:**
    - `text`: İşlenecek, gösterilecek veya aranacak metin içeriği.

---

ti_settings.py

Modül Tanımı: Uygulama ayarlarının, splitter pozisyonlarının ve alarm ses dosyalarının yönetimini sağlayan mixin modülü.

`SettingsMixin` (Sınıf)

Uygulama ayarlarını ve pencere durumlarını yöneten mixin sınıfı.

- `alarm_dosyalarini_bul(self)`
  - **İşlevi:** Timer-data klasöründeki tüm .mp3 dosyalarını bulur
- `ayarlari_kaydet(self)`
  - **İşlevi:** Pencere boyutu, konumu, varsayılan süre, favoriler ve geçmiş verilerini JSON dosyasına kalıcı olarak yazar.
- `splitter_pozisyonu_kaydet(self, pos, index)`
  - **İşlevi:** Tüm splitter pozisyonlarını kaydet.
  - **Parametreler:**
    - `pos`: Fare imlecinin tıklandığı yerel koordinat noktası (QPoint).
    - `index`: İşlem yapılacak öğenin dizin indeksi (int).
- `splitter_pozisyonu_yukle(self)`
  - **İşlevi:** Tüm splitter pozisyonlarını yükle.
- `ayarlari_yukle(self)`
  - **İşlevi:** Ayarları ve aktif zamanlayıcıları yükle.
- `favori_yedekle(self)`

- **İşlevi:** Favori zamanlayıcı kayıtlarını timer-data klasörüne JSON formatında yedekler.

- `favori_yedekten_geri_don(self)`
  - **İşlevi:** Seçilen JSON yedek dosyasındaki favorileri önizleme penceresinde listeler ve onaylanırsa geri yükler.

---

ti_stopwatch.py

Modül Tanımı: Kronometre kartlarının yönetimi, tur kayıtları ve sağ tık menülerini sağlayan mixin modülü.

`KronometreMixin` (Sınıf)

Kronometre kartlarının yönetimi ve etkileşimlerini sağlayan mixin sınıfı.

- `yeni_kronometre_baslat(self)`
  - **İşlevi:** Yeni kronometre başlatmak için isim soran dialog açar.
- `kronometre_widget_olustur(self, kronometre)`
  - **İşlevi:** Kronometre için widget oluşturur.
  - **Parametreler:**
    - `kronometre`: Kronometre veya zamanlayıcı nesnesi.
- `kronometre_durdur_devam(self, kronometre_id)`
  - **İşlevi:** Kronometreyi durdurur veya devam ettirir.
  - **Parametreler:**
    - `kronometre_id`: Kronometrenin benzersiz kimlik numarası.
- `kronometre_kapat(self, kronometre_id)`
  - **İşlevi:** Kronometreyi kapatır.
  - **Parametreler:**
    - `kronometre_id`: Kronometrenin benzersiz kimlik numarası.
- `kronometre_tur_ekle(self, kronometre_id)`
  - **İşlevi:** Kronometreye tur ekler.
  - **Parametreler:**
    - `kronometre_id`: Kronometrenin benzersiz kimlik numarası.
- `kronometre_sag_tik_menu(self, position, kronometre_id)`
  - **İşlevi:** Kronometre için sağ tık context menüsü.

    - **Parametreler:**
        - `position`: Fare tıklamasının gerçekleştiği koordinat noktası (QPoint).
        - `kronometre_id`: Kronometrenin benzersiz kimlik numarası.

- `kronometre_sifirla(self, kronometre_id)`
    - **İşlevi:** Kronometreyi sıfırlar.
    - **Parametreler:**
        - `kronometre_id`: Kronometrenin benzersiz kimlik numarası.

- `kronometre_isim_degistir(self, kronometre_id)`
    - **İşlevi:** Kronometre ismini değiştirir.
    - **Parametreler:**
        - `kronometre_id`: Kronometrenin benzersiz kimlik numarası.

---

ti_timer_widgets.py

Modül Tanımı: Aktif zamanlayıcı widget kartlarının oluşturulması, düzenlenmesi ve kullanıcı etkileşimlerini yöneten mixin modülü.

`TimerWidgetsMixin` (Sınıf)

Zamanlayıcı widget kartlarının oluşturulması ve yönetimini sağlayan mixin sınıfı.

- `zamanlayici_widget_olustur(self, zamanlayici)`
    - **İşlevi:** Belirtilen zamanlayıcı parametreleriyle yeni bir ZamanlayiciWidget kartı üretir ve ana alana yerleştirir.
    - **Parametreler:**
        - `zamanlayici`: Zamanlayıcı yapılandırma verilerini içeren nesne veya sözlük.

- `handle_timer_double_click_event(self, timer_object_name_str)`
    - **İşlevi:** Çift tıklanan zamanlayıcı için düzenleme diyalogunu açar.
    - **Parametreler:**
        - `timer_object_name_str`: Zamanlayıcı widget nesnesinin Qt nesne adı.

- `duzenle_aktif_zamanlayici(self, timer_id)`
    - **İşlevi:** Çalışmakta olan bir zamanlayıcının kalan süresini, etiketini veya alarm sesini düzenleme diyaloğunu açar.
    - **Parametreler:**
        - `timer_id`: Düzenlenecek zamanlayıcının tekil kimlik değeri.

- `guncelle_zamanlayici_widget_arayuzu(self, zamanlayici)`

    - **İşlevi:** Belirli bir zamanlayıcının arayüzdeki widget'larını günceller.
    - **Parametreler:**
        - `zamanlayici`: Zamanlayıcı yapılandırma veya widget nesnesi.

- `süre_Degistir(self, zamanlayici_id)`
    - **İşlevi:** Aktif zamanlayıcının süresini hızlı butonlarla veya kullanıcı girdisiyle artırıp azaltır.
    - **Parametreler:**
        - `zamanlayici_id`: Süresi değiştirilecek zamanlayıcının tekil kimlik değeri.
- `kullanicidan_sayi_al(self)`
    - **İşlevi:** Kullanıcıdan sayısal süre girdisi almak üzere standart QInputDialog iletişim penceresini sunar.

---

ti_ui_builder.py

Modül Tanımı: Ana arayüz düzenini, kenar panellerini (favoriler, geçmiş) ve kısayolları inşa eden mixin modülü.

`UIBuilderMixin` (Sınıf)

Kullanıcı arayüzü bileşenlerini ve yan panelleri oluşturan mixin sınıfı.

- `kisayol_tuslari_ayarla(self)`
    - **İşlevi:** Klavye kısayollarını ayarla.
- `panelleri_kapat(self)`
    - **İşlevi:** ESC tuşu ile açık panelleri kapat.
- `arayuzu_olustur(self)`
    - **İşlevi:** Ana pencerenin tüm buton, girdi kutusu, kaydırma alanı ve menü bileşenlerini oluşturur.

---

timer_file_analyzer.py

Modül Tanımı: Zamanlayıcı geçmiş ve ayar verilerini tablo formatında listeleyen, filtreleyen ve dışa aktaran analiz penceresi.

`DosyaAnaliziPenceresi` (Sınıf)

DosyaAnaliziPenceresi sınıfı, ilgili işlevleri ve süreçleri yönetir.

- `__init__(self, parent)`
  - **İşlevi:** DosyaAnaliziPenceresi sınıfını ve gerekli bileşenlerini ilklendirir.
  - **Parametreler:**
    - `parent`: Üst pencere veya taşıyıcı arayüz bileşeni referansı.
- `klasor_sec(self)`
  - **İşlevi:** Klasör seçme diyaloğunu göster.
- `klasore_git(self, item)`
  - **İşlevi:** Tabloda seçilen klasöre git.
  - **Parametreler:**
    - `item`: Seçilen liste veya tablo öğesi referansı.
- `ust_klasore_git(self)`
  - **İşlevi:** Bir üst klasöre git.
- `boyut_formatla(self, boyut)`
  - **İşlevi:** Boyutu okunabilir formata dönüştür.
  - **Parametreler:**
    - `boyut`: Dosya boyutu veya görsel boyut piksel değeri.
- `klasor_boyutu_ve_dosya_sayisi_hesapla(self, klasor_yolu, ilerleme_dialog)`
  - **İşlevi:** Klasörün toplam boyutunu ve dosya sayısını hesapla.
  - **Parametreler:**
    - `klasor_yolu`: Taranacak veya işlem yapılacak klasörün adı/konumu.
    - `ilerleme_dialog`: İşlem sırasında gösterilen ilerleme diyalog penceresi.
- `mevcut_klasor_dosyalarini_analiz_et(self)`
  - **İşlevi:** Mevcut klasördeki dosyaların (alt klasörler hariç) toplam boyutunu hesapla.
- `klasor_analizi_yap(self)`
  - **İşlevi:** Mevcut klasörün analizini yap ve tabloda göster.
- `tablo_satiri_ekle(self, satir, isim, boyut, dosya_sayisi, arka_plan_rengi, renk)`
  - **İşlevi:** Tabloya yeni bir satır ekler.
  - **Parametreler:**
    - `satir`: Tahta veya ızgara üzerindeki satır indeksi.
    - `isim`: Kayıt, profil veya dosya adı.
    - `boyut`: Dosya boyutu veya görsel boyut piksel değeri.
    - `dosya_sayisi`: Taranan veya bulunan toplam dosya adedi.
    - `arka_plan_rengi`: Bileşenin arka plan renk kodu (hex/RGB).

    - `renk`: Çizim veya metin rengi (QColor/string).
- `tablo_disariya_aktar(self)`
    - **İşlevi:** Tablo verilerini txt dosyasına formatlanmış olarak aktar.

---

timer_formatter.py

Modül Tanımı: Süre, zaman damgası ve metin biçimlendirme işlemlerini yürüten yardımcı fonksiyonlar modülü.

Bağımsız Fonksiyonlar

- `format_time(total_seconds)`
    - **İşlevi:** Kalan süreyi uygun formatta döndürür.
    - **Parametreler:**
        - `total_seconds`: Biçimlendirilecek toplam saniye miktarı.
- `get_current_datetime_string(format_str)`
    - **İşlevi:** Şu anki tarih ve saati belirtilen formatta bir string olarak döndürür.
    - **Parametreler:**
        - `format_str`: Zaman biçimlendirme kalıp dizgesi.

---

timer_helpers.py

Modül Tanımı: Dosya işlemleri, ses arama, tema ve simge yönetimi gibi genel yardımcı işlevleri sağlayan modül.

`TimerHelpers` (Sınıf)

Zamanlayıcı yardımcı metodları sınıfı.

- `__init__(self, parent_app)`
    - **İşlevi:** TimerHelpers sınıfını ve gerekli bileşenlerini ilklendirir.
    - **Parametreler:**
        - `parent_app`: Ana uygulama penceresi örneği referansı.
- `favoriye_ekle(self, gecmis_indeks)`
    - **İşlevi:** Seçilen geçmiş kaydını favorilere ekle.

    - **Parametreler:**
        - `gecmis_indeks`: Favoriye dönüştürülecek geçmiş kaydının liste indeksi.

- `favorileri_goster(self)`
    - **İşlevi:** Favori listesini göster/gizle.
- `favori_listesini_guncelle(self)`
    - **İşlevi:** Favori listesini güncelle.
- `favori_secimi_degisti(self)`
    - **İşlevi:** Favori listesindeki seçim değiştiğinde silme düğmesini etkinleştir/devre dışı bırak.
- `favoriden_sil(self, favori_indeks)`
    - **İşlevi:** Seçilen favoriyi sil.
    - **Parametreler:**
        - `favori_indeks`: Silinecek favori kaydının liste indeksi.
- `secilen_favorileri_sil(self)`
    - **İşlevi:** Seçilen favorileri sil.
- `favori_zamanlayici_baslat(self, item)`
    - **İşlevi:** Favori zamanlayıcıyı başlat.
    - **Parametreler:**
        - `item`: Seçilen favori liste öğesi referansı (QListWidgetItem).
- `favori_duzenle(self, favori_indeks)`
    - **İşlevi:** Seçilen favoriyi düzenle.
    - **Parametreler:**
        - `favori_indeks`: Düzenlenecek favori kaydının liste indeksi.
- `gecmis_secimi_degisti(self)`
    - **İşlevi:** Geçmiş listesindeki seçim değiştiğinde silme düğmesini etkinleştir/devre dışı bırak.
- `secilen_gecmisi_sil(self)`
    - **İşlevi:** Seçilen geçmiş kayıtlarını sil.
- `gecmisi_goster(self, force_refresh_only_if_visible)`
    - **İşlevi:** Geçmiş listesini göster/gizle.
    - **Parametreler:**
        - `force_refresh_only_if_visible`: Yalnızca pencere ekranda görünür durumdaysa yenileme yapılmasını sağlayan bayrak.

- `gecmis_zamanlayici_baslat(self, item)`
  - **İşlevi:** Geçmiş listesinden zamanlayıcı başlat (çift tıklama) - Doğrudan başlat.
  - **Parametreler:**
    - `item`: Seçilen geçmiş liste öğesi referansı (QListWidgetItem).
- `ayarlari_kaydet(self)`
  - **İşlevi:** Ayarları ve aktif zamanlayıcıları kaydet.
- `loglari_goster_DEPRECIATED(self)`
  - **İşlevi:** Logları göster.

Bağımsız Fonksiyonlar

- `show_toast(parent, msgTime, message, duration)`
  - **İşlevi:** Ekranın sağ alt köşesinde belirtilen süre boyunca kaybolan geçici bildirim balonu görüntüler.
  - **Parametreler:**
    - `parent`: Üst pencere veya taşıyıcı arayüz bileşeni referansı.
    - `msgTime`: Bildirimin oluşturulma zaman damgası metni.
    - `message`: Kullanıcıya gösterilecek durum veya bildirim iletisi.
    - `duration`: Bildirimin ekranda kalma süresi (milisaniye cinsinden).

---

timer_logger.py

Modül Tanımı: Mikro saniye hassasiyetli loglama motoru ve log filtreleme/görüntüleme diyalog penceresi.

`MicrosecondFormatter` (Sınıf)

MicrosecondFormatter sınıfı, ilgili işlevleri ve süreçleri yönetir.

- `formatTime(self, record, datefmt)`
  - **İşlevi:** Log kayıtlarının zaman damgasını mikro saniye (.mmm) hassasiyetinde biçimlendirir.
  - **Parametreler:**
    - `record`: Biçimlendirilecek LogRecord nesnesi.
    - `datefmt`: İsteğe bağlı tarih biçim dizgesi.

`LogViewerDialog` (Sınıf)

LogViewerDialog sınıfı, ilgili işlevleri ve süreçleri yönetir.

- `center_window(self)`
  - **İşlevi:** Pencereyi masaüstü ekranının merkez koordinatına yerleştirir.
- `__init__(self, parent, log_file_path)`
  - **İşlevi:** LogViewerDialog sınıfını ve gerekli bileşenlerini ilklendirir.
  - **Parametreler:**
    - `parent`: Üst pencere veya taşıyıcı arayüz bileşeni referansı.
    - `log_file_path`: Log dosyasının adı veya konumu.
- `setup_ui(self)`
  - **İşlevi:** Pencerenin tüm grafiksel kullanıcı arayüzü bileşenlerini, yerleşim düzenlerini ve sinyal bağlantılarını kurar.
- `show_search_context_menu(self, pos)`
  - **İşlevi:** Arama kutusu üzerinde sağ tıklandığında sık kullanılan arama şablonları menüsünü açar.
  - **Parametreler:**
    - `pos`: Fare imlecinin tıklandığı yerel koordinat noktası (QPoint).
- `select_favorite_search(self, fav_text)`
  - **İşlevi:** Menüden seçilen favori arama sorgusunu filtre kutusuna uygular.
  - **Parametreler:**
    - `fav_text`: Seçilen favori arama sorgu metni.
- `add_search_to_favorites(self)`
  - **İşlevi:** Arama kutusundaki metni log_favori_search.txt dosyasına ekler.
- `save_logs(self)`
  - **İşlevi:** QTextEdit içindeki mevcut içeriği log dosyasına kaydeder.
- `load_logs(self)`
  - **İşlevi:** Log dosyasını okuyarak kayıtları ayrıştırır ve tabloya doldurur.
- `clear_search(self)`
  - **İşlevi:** Arama alanını temizler ve filtreleri uygular.
- `apply_filter(self)`
  - **İşlevi:** Kullanıcının arama metnine ve seçtiği log seviyesine göre log satırlarını süzer.

Bağımsız Fonksiyonlar

- `setup_logging(log_file_path)`

    - **İşlevi:** Initialize logging system with proper configuration.
    - **Parametreler:**
        - `log_file_path`: Log dosyasının adı veya konumu.

- `record_log(mesaj, level)`
    - **İşlevi:** Log messages with specified level.
    - **Parametreler:**
        - `mesaj`: Kullanıcıya gösterilecek uyarı veya bilgi mesajı.
        - `level`: Mevcut oyun zorluk seviyesi.

- `view_filtered_logs(log_file_path)`
    - **İşlevi:** Open a window to view and filter log files.
    - **Parametreler:**
        - `log_file_path`: Log dosyasının adı veya konumu.

---

timer_reminder_system.py

Modül Tanımı: Tekrarlı ve tekil hatırlatıcı veri modelini ve tarih/saat hesaplama mantığını yöneten çekirdek sistem.

`Hatirlatici` (Sınıf)

Hatirlatici sınıfı, ilgili işlevleri ve süreçleri yönetir.

- `__init__(self, id, tarih, saat, not_metni, yapildi, olusturma_zamani, tekrarlama_tipi, tekrarlama_araligi, son_tekrar_tarihi, hafta_gunu, ertelendi)`
    - **İşlevi:** Hatirlatici sınıfını ve gerekli bileşenlerini ilklendirir.
    - **Parametreler:**
        - `id`: Kaydın tekil kimlik numarası.
        - `tarih`: Hedef tarih bilgisi.
        - `saat`: Hedef saat bilgisi.
        - `not_metni`: Hatırlatıcı mesaj veya görev metni.
        - `yapildi`: Görevin tamamlanma durumunu belirten mantıksal değer.
        - `olusturma_zamani`: Kaydın ilk oluşturulduğu tarih ve saat (datetime).
        - `tekrarlama_tipi`: Tekrarlama periyodu türü ("yok", "gun", "hafta", "ay").
        - `tekrarlama_araligi`: Tekrarlama aralığı katsayısı (her X günde/haftada bir).
        - `son_tekrar_tarihi`: En son tetiklenen tekrarın tarihi.
        - `hafta_gunu`: Haftalık tekrarlar için hedef gün indeksi (0: Pazartesi - 6: Pazar).
        - `ertelendi`: Hatırlatıcının ertelenip ertelenmediğini belirten bayrak.

- `get_datetime(self)`

    - **İşlevi:** Hatırlatıcının tarih ve saatini datetime objesi olarak döndürür.
- `is_gecmis(self)`
    - **İşlevi:** Hatırlatıcının zamanı geçmiş mi kontrol eder.
- `get_sonraki_tekrar_tarihi(self)`
    - **İşlevi:** Bir sonraki tekrar tarihini hesaplar.
- `sonraki_tekrari_olustur(self)`
    - **İşlevi:** Tekrarlayan hatırlatıcı için bir sonraki örneği oluşturur.
- `to_dict(self)`
    - **İşlevi:** Hatırlatıcı nesnesi özelliklerini JSON ile uyumlu Python sözlük yapısına dönüştürür.
- `from_dict(cls, data)`
    - **İşlevi:** Sözlük yapısındaki verileri çözümleyerek yeni bir Hatirlatici nesnesi oluşturur.
    - **Parametreler:**
        - `data`: İşlenecek veya analiz edilecek veri kümesi veya nesnesi.

`HatirlaticiDialog` (Sınıf)

HatirlaticiDialog sınıfı, ilgili işlevleri ve süreçleri yönetir.

- `__init__(self, parent, hatirlatici)`
    - **İşlevi:** HatirlaticiDialog sınıfını ve gerekli bileşenlerini ilklendirir.
    - **Parametreler:**
        - `parent`: Üst pencere veya taşıyıcı arayüz bileşeni referansı.
        - `hatirlatici`: İşlem yapılacak Hatirlatici nesnesi referansı.
- `setup_ui(self)`
    - **İşlevi:** Pencerenin tüm grafiksel kullanıcı arayüzü bileşenlerini, yerleşim düzenlerini ve sinyal bağlantılarını kurar.
- `tekrarlama_degisti(self)`
    - **İşlevi:** Tekrarlama seçimi değiştiğinde arayüzü günceller.
- `populate_saat_combo(self)`
    - **İşlevi:** 30 dakikalık aralıklarla saat seçeneklerini doldur.
- `load_hatirlatici(self)`
    - **İşlevi:** Mevcut hatırlatıcı bilgilerini forma yükle.

- `get_values(self)`
  - **İşlevi:** Form değerlerini döndür.

`HatirlaticiBildirimDialog` (Sınıf)

HatirlaticiBildirimDialog sınıfı, ilgili işlevleri ve süreçleri yönetir.

- `__init__(self, parent, hatirlatici)`
  - **İşlevi:** HatirlaticiBildirimDialog sınıfını ve gerekli bileşenlerini ilklendirir.
  - **Parametreler:**
    - `parent`: Üst pencere veya taşıyıcı arayüz bileşeni referansı.
    - `hatirlatici`: İşlem yapılacak Hatirlatici nesnesi referansı.
- `setup_ui(self)`
  - **İşlevi:** Pencerenin tüm grafiksel kullanıcı arayüzü bileşenlerini, yerleşim düzenlerini ve sinyal bağlantılarını kurar.
- `yapildi_clicked(self)`
  - **İşlevi:** Kullanıcı görevi tamamladığını belirttiğinde hatırlatıcıyı tamamlandı olarak işaretler ve pencereyi kapatır.
- `yapilmayacak_clicked(self)`
  - **İşlevi:** Kullanıcı görevin yapılmayacağını belirttiğinde durumu işler ve pencereyi kapatır.

---

timer_reminder_ui.py

Modül Tanımı: Hatırlatıcı ekleme, düzenleme, listeleme ve alarm bildirim pencerelerini sunan arayüz modülü.

`HatirlaticiManager` (Sınıf)

Hatırlatıcı sisteminin UI ve yönetim işlevlerini sağlayan yardımcı sınıf.

- `__init__(self, uygulama)`
  - **İşlevi:** uygulama: ZamanlayiciUygulamasi örneği.
  - **Parametreler:**
    - `uygulama`: Ana uygulama (ZamanlayiciUygulamasi) örneği referansı.
- `yeni_hatirlatici_ekle(self)`
  - **İşlevi:** Yeni hatırlatıcı ekleme diyaloğunu aç.

- `hatirlatici_sag_tik_menu(self, position)`
  - **İşlevi:** Tüm hatırlatıcılar listesi için sağ tık menüsü.
  - **Parametreler:**
    - `position`: Fare tıklamasının gerçekleştiği koordinat noktası (QPoint).
- `get_hatirlatici_from_sorted_list(self, sorted_index)`
  - **İşlevi:** Sıralı listeden gerçek hatırlatıcı objesini döndür - ID tabanlı.
  - **Parametreler:**
    - `sorted_index`: Sıralı listedeki hedef öğe indeksi.
- `hatirlatici_duzenle_by_object(self, hatirlatici)`
  - **İşlevi:** Verilen hatırlatıcı nesnesini düzenlemek üzere diyalog penceresini açar ve yapılan değişiklikleri sisteme kaydeder.
  - **Parametreler:**
    - `hatirlatici`: Düzenlenecek Hatirlatici nesnesi referansı.
- `hatirlatici_sil_by_object(self, hatirlatici)`
  - **İşlevi:** Hatırlatıcı objesini kullanarak silme.
  - **Parametreler:**
    - `hatirlatici`: İşlem yapılacak Hatirlatici nesnesi referansı.
- `gecmis_hatirlatici_sag_tik_menu(self, position)`
  - **İşlevi:** Geçmiş hatırlatıcılar listesi için sağ tık menüsü.
  - **Parametreler:**
    - `position`: Fare tıklamasının gerçekleştiği koordinat noktası (QPoint).
- `get_hatirlatici_from_gecmis_list(self, gecmis_index)`
  - **İşlevi:** Geçmiş listesinden gerçek hatırlatıcı objesini döndür.
  - **Parametreler:**
    - `gecmis_index`: Geçmiş listesindeki kaydın sıra numarası.
- `hatirlatici_yapildi_isaretle(self, item)`
  - **İşlevi:** Hatırlatıcıyı yapıldı olarak işaretle.
  - **Parametreler:**
    - `item`: Seçilen liste veya tablo öğesi referansı.
- `hatirlatici_listelerini_guncelle(self, kalan_sure_guncelle)`
  - **İşlevi:** Hatırlatıcı listelerini güncelle.
  - **Parametreler:**
    - `kalan_sure_guncelle`: Kalan sürenin ekranda güncellenip güncellenmeyeceği bayrağı.
- `kalan_sure_metni_olustur(self, kalan_sure)`

    - **İşlevi:** Kalan süreyi formatlı metin olarak döndürür.
    - **Parametreler:**
        - `kalan_sure`: Zamanlayıcının bitimine kalan süre (saniye).

- `tekrarli_hatirlaticilari_guncelle(self)`
    - **İşlevi:** Geçmiş olan tekrarlı hatırlatıcıları otomatik olarak sonraki tekrar tarihine güncelle.

- `hatirlatici_kontrol(self)`
    - **İşlevi:** Zamanı gelen hatırlatıcıları kontrol et.

- `hatirlatici_bildirim_goster(self, hatirlatici)`
    - **İşlevi:** Hatırlatıcı bildirimini göster.
    - **Parametreler:**
        - `hatirlatici`: İşlem yapılacak Hatirlatici nesnesi referansı.

- `hatirlatici_ses_cal(self, ses_dosyasi)`
    - **İşlevi:** Hatırlatıcı için ses çal.
    - **Parametreler:**
        - `ses_dosyasi`: Çalınacak ses veya alarm dosyasının adı.

---

ti_reminder_notification.py

Modül Tanımı: Hatırlatıcı bildirim penceresi oluşturma, erteleme ve ses çalma işlevlerini yöneten yardımcı modül.

Bağımsız Fonksiyonlar

- `hatirlatici_bildirim_goster(manager, hatirlatici)`
    - **İşlevi:** Hatırlatıcı bildirim penceresini görüntüler, kullanıcının yapıldı veya ertele seçimlerini işler.
    - **Parametreler:**
        - `manager`: HatirlaticiManager nesnesi referansı.
        - `hatirlatici`: Bildirimi gösterilecek Hatirlatici nesnesi referansı.

- `hatirlatici_ses_cal(manager, ses_dosyasi)`
    - **İşlevi:** Hatırlatıcı uyarı sesini çalar.
    - **Parametreler:**
        - `manager`: HatirlaticiManager nesnesi referansı.
        - `ses_dosyasi`: Çalınacak ses veya alarm dosyasının adı.